Lab 02: Ethics, Law, and Responsible Data Collection

INFO 4617/5617 · Week 2 · Chapter 2, Ethics, Law, and Responsible Data Collection

Today we build the habits the chapter asks for before any collection: a User-Agent that says who we are, a `robots.txt` read properly, a pause between requests, and one function that does all of it. Our case is the news business, where the chapter says `robots.txt` has become a front line: we read the New York Times's file line by line, then ask six newsrooms which AI crawlers they let in. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it.

Before class: open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent.

On Friday, run every cell above where we stopped (**Run → Run All Above Selected Cell**).

> **Instructor key.** Every cell is filled in and was run against the live sites on 2026-10-21, in Python 3.14.8, following the textbook at commit `d9306b7`. `robots.txt` files change without notice and have no dated versions to ask for, so the counts and the allowed/blocked tables in Steps 2, 3, 5 and 7 may differ on a rerun. The 403s in Step 7 depend on the network a request comes from: from campus, a site may answer differently. Python before 3.14.5 parses `robots.txt` by older rules (Step 3), so it gives different answers for some rows. Every cell was run live.

Step 0 — Set up (5 minutes)

Import what we need, then define the User-Agent every request will send. The chapter's Responsible Request Headers section asks for a name, a version, and a way to reach you, in the form Wikimedia's User-Agent policy gives: `<client name>/<version> (<contact information>)`. We'll also parse `robots.txt` files with `urllib.robotparser`, part of Python itself, so print which Python you have: in Step 3 it changes the answers.

```
# Import sys, time, requests, pandas (as pd), matplotlib.pyplot (as plt), and RobotFileParser
 ↪  from urllib.robotparser
# Define HEADERS: a dict whose "User-Agent" is "WebDataScience/1.0 (your-email@colorado.edu)",
 ↪  with your email
# Print HEADERS, and the Python version (the first word of sys.version)
import sys
import time

import requests
import pandas as pd
import matplotlib.pyplot as plt
from urllib.robotparser import RobotFileParser

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
print(HEADERS)
print("Python", sys.version.split()[0])
```

```
{'User-Agent': 'WebDataScience/1.0 (your-email@colorado.edu)'}
Python 3.14.8
```

💬 Discuss

- In week 1 we sent this header to Wikipedia because the chapter told us to, and it promised to explain why later. Who reads it, and when?
- Every request you send this week carries your email address. What does that commit you to?
- Nothing stops a script from copying a browser's User-Agent instead. Why does the chapter ask you to name yourself?

Step 1 — What a server sees (6 minutes)

A server can't see your code, only your request. httpbin.org/headers answers every request with the headers it received, so we can read our request the way a server does. Ask it twice, once with no headers and once with `headers=HEADERS`, and compare the User-Agents. Then ask for week 1's Wikipedia article with no headers at all. These are the lab's only requests sent without `HEADERS`, on purpose.

```
# Request https://httpbin.org/headers twice, once with no headers and once with HEADERS;
#   from each .json(), print the "User-Agent" inside "headers"
# Then request week 1's article, https://en.wikipedia.org/wiki/University_of_Colorado_Boulder,
#   with no headers, and print the status code
ECHO = "https://httpbin.org/headers"
print(requests.get(ECHO, timeout=30).json()["headers"]["User-Agent"])
print(requests.get(ECHO, headers=HEADERS, timeout=30).json()["headers"]["User-Agent"])

response = requests.get("https://en.wikipedia.org/wiki/University_of_Colorado_Boulder",
 ↪  timeout=30)
print(response.status_code)
```

```
python-requests/2.34.2
```

```
WebDataScience/1.0 (your-email@colorado.edu)
```

```
403
```

💬 Discuss

- In week 1 Wikipedia refused `requests`' default User-Agent with a 403. Is refusing anonymous clients fair for a site that wants to be read?
- Print the whole `headers` dictionary the echo sent back. What else did the server learn about us besides the User-Agent?
- The default names the library and its version but not you. Whom can a site operator contact about a misbehaving `python-requests` client?

Step 2 — Read a robots.txt (7 minutes)

A `robots.txt` file lives at the root of a site and says which parts automated agents may fetch: `User-agent` lines name the crawlers a group of rules is for, then `Disallow` and `Allow` lines give paths (chapter, Technical Norms). The chapter compares the New York Times's file with Wikipedia's and Reddit's. Fetch the Times's file with `HEADERS`, check the status with `raise_for_status()`, and split it into lines. Before any rules, it opens with a notice; print it. Then list the crawlers the file names.

```
# Fetch https://www.nytimes.com/robots.txt with HEADERS; print the status code, and stop if it
 ↪  isn't 200
# Split the text into lines, and print the comment lines (they start with "#") among the first 15
# Collect the name after every "User-agent:" line; print how many names there are, and how many
 ↪  are different
response = requests.get("https://www.nytimes.com/robots.txt", headers=HEADERS, timeout=30)
print(response.status_code)
response.raise_for_status()

lines = response.text.splitlines()
for line in lines[:15]:
    if line.startswith("#"):
        print(line)

agents = [line.split(":", 1)[1].strip() for line in lines if
 ↪  line.lower().startswith("user-agent")]
```

```
print(len(agents), "User-agent lines,", len(set(agents)), "different crawlers, such as",
 ↪  agents[:8])
```

```
200
# New York Times content is made available for your personal, non-commercial
# use subject to our Terms of Service here:
# https://help.nytimes.com/hc/en-us/articles/115014893428-Terms-of-Service.
# Use of any device, tool, or process designed to data mine or scrape the content
# using automated means is prohibited without prior written permission from
# The New York Times Company.  Prohibited uses include but are not limited to:
# (1) text and data mining activities under Art. 4 of the EU Directive on Copyright in
# the Digital Single Market;
# (2) the development of any software, machine learning, artificial intelligence (AI),
# and/or large language models (LLMs);
# (3) creating or providing archived or cached data sets containing our content to others; and/or
# (4) any commercial purposes.
# Contact https://nytlicensing.com/contact/ for assistance.
69 User-agent lines, 68 different crawlers, such as ['*', 'Googlebot', 'Googlebot',
 ↪  'AliyunSecBot', 'Amazonbot', 'anthropic-ai', 'Applebot-Extended', 'archive.org_bot']
```

💬 Discuss

- The notice says what the Times forbids, but a parser can't read it. Which of its four prohibited uses could a `Disallow` line express, and which not?
- The notice points to the Terms of Service. After hiQ v. LinkedIn, where does the legal risk sit for someone who collects public Times pages: the CFAA, or the contract?
- Which crawlers in the list do you recognize? What does naming dozens of them, one by one, tell you about the site's view of automated access?

Step 3 — Ask the parser (11 minutes)

Reading hundreds of lines by eye doesn't scale, so hand them to a `RobotFileParser` with `parse()`, and ask it questions with `can_fetch(agent, url)`. We already have the lines from Step 2, so this sends no request. Ask about four pages, for two agents: `"*"`, the group for every crawler the file doesn't name (that includes us), and `"GPTBot"`, OpenAI's crawler.

Since Python 3.14.5, the parser follows RFC 9309, the standard that Google's crawlers follow: when several rules match a path, the longest one wins. Older versions obeyed the first matching rule in the file. If Step 0 printed an older Python, one row below will come out differently; the microsprint shows which.

```
# Make a RobotFileParser and parse() Step 2's lines (no second request)
# Make PAGES: the Times's home page, /section/technology, /search?query=robots, and /svc/wordle
# Build a DataFrame with one row per page and one column per agent ("*" and "GPTBot") holding
 ↪  can_fetch(agent, page)
rp = RobotFileParser()
rp.parse(lines)

PAGES = ["https://www.nytimes.com/",
         "https://www.nytimes.com/section/technology",
         "https://www.nytimes.com/search?query=robots",
         "https://www.nytimes.com/svc/wordle"]
answers = pd.DataFrame({agent: [rp.can_fetch(agent, page) for page in PAGES] for agent in ["*",
 ↪  "GPTBot"]},
                       index=PAGES)
print(answers)
```

```
                                                 *  GPTBot
```

```
https://www.nytimes.com/                        True   False
https://www.nytimes.com/section/technology      True   False
https://www.nytimes.com/search?query=robots     False  False
https://www.nytimes.com/svc/wordle              True   False
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Do: check one of the parser's answers by hand. In the group for `"*"`, find every `Allow` or `Disallow` line whose path is the start of `/svc/wordle`, with its line number. **Share:** the matching lines, the one `can_fetch()` obeyed for `"*"`, and why that one.

```
# Microsprint: the group for "*" runs from the line "User-agent: *" to the next empty line; find
 ↪  both with lines.index()
# Loop over that group with line numbers; print each Allow or Disallow line whose path (after the
 ↪  colon)
#   isn't empty and is the start of "/svc/wordle"
start = lines.index("User-agent: *")
end = lines.index("", start)
path = "/svc/wordle"
for number, line in enumerate(lines[start:end], start=start + 1):
    if line.startswith(("Allow:", "Disallow:")):
        rule = line.split(":", 1)[1].strip()
        if rule and path.startswith(rule):
            print(number, line)
```

```
70 Disallow: /svc
75 Allow: /svc/wordle
```

💬 Discuss

- Two rules match `/svc/wordle`, and before Python 3.14.5 the parser obeyed the other one. Which answer did the Times mean, and how can you tell from the file?
- Week 1 said an environment records which Python an analysis ran on. Why did that just matter for a `robots.txt` audit run last year?
- `can_fetch()` says GPTBot may not fetch the home page. Do the rules for `"*"` or for `GPTBot` apply to us?

Step 4 — The trap in `rp.read()` (7 minutes)

`RobotFileParser` can also fetch the file itself, with `read()`, and almost every tutorial does it that way. The chapter's callout, Why not `rp.read()`?, explains why we don't. Watch it happen on Wikipedia: let `read()` fetch the file, then ask about an ordinary article. Then send the request `read()` sends, through `urllib.request` with its defaults, and look at its User-Agent and the answer it gets. Here we break our rule about `HEADERS` on purpose.

```
# Make a RobotFileParser for https://en.wikipedia.org/robots.txt and call read(); print
 ↪  disallow_all,
#   and can_fetch("*", ...) for the chapter's article
 ↪  https://en.wikipedia.org/wiki/Python_(programming_language)
# Import urllib.request and urllib.error; print the default User-Agent
 ↪  (urllib.request.build_opener().addheaders)
# Open the same robots.txt with urllib.request.urlopen(); catch urllib.error.HTTPError, and print
 ↪  its code and reason
wiki = RobotFileParser("https://en.wikipedia.org/robots.txt")
wiki.read()
print("disallow_all:", wiki.disallow_all)
print("can_fetch:", wiki.can_fetch("*",
 ↪  "https://en.wikipedia.org/wiki/Python_(programming_language)"))
```

```
import urllib.request
import urllib.error

print(urllib.request.build_opener().addheaders)
try:
    urllib.request.urlopen("https://en.wikipedia.org/robots.txt", timeout=30)
except urllib.error.HTTPError as error:
    print(error.code, error.reason)
```

```
disallow_all: True
can_fetch: False
[('User-agent', 'Python-urllib/3.14')]
```

```
403 Forbidden
```

💬 Discuss

- Had we stopped after `read()`, what would we have concluded about Wikipedia, and would anything have warned us?
- In week 1 a 4xx status code meant something was wrong with our request, and we could see it. Where did this 403 go?
- After Van Buren, the CFAA is about gates you aren't entitled to pass. Is a `Disallow` line a gate, and does a 403 change your answer?

Step 5 — Which AI crawlers may come in? (13 minutes)

The chapter says the lawsuits over AI training have changed the web: publishers "added AI-specific directives to `robots.txt`, and signed exclusive licensing deals" (chapter, The Current Landscape). Let's check that across one sector. Write `crawler_check(site)`: fetch the site's `robots.txt` with `HEADERS`, parse it, and ask `can_fetch()` whether each agent in `AGENTS` may fetch the home page. Run it on six newsrooms, two of them in Colorado, with `time.sleep(1)` between sites, as the chapter's comparison should have.

```
# Define AGENTS: "*", "Googlebot", "GPTBot", "ClaudeBot", "CCBot", "Google-Extended",
 ↪  "PerplexityBot"
# Define crawler_check(site): fetch f"https://{site}/robots.txt" with HEADERS, raise_for_status,
 ↪  parse the lines,
#   and return a dict: for each agent, can_fetch(agent, f"https://{site}/")
# Run it on each site in NEWS, sleeping 1 second between sites; make a DataFrame, one row per
 ↪  site,
#   and print it with each True or False written as "allowed" or "blocked" (DataFrame.map)
AGENTS = ["*", "Googlebot", "GPTBot", "ClaudeBot", "CCBot", "Google-Extended", "PerplexityBot"]
NEWS = ["www.nytimes.com", "www.washingtonpost.com", "www.wsj.com",
        "www.theguardian.com", "www.denverpost.com", "coloradosun.com"]

def crawler_check(site):
    response = requests.get(f"https://{site}/robots.txt", headers=HEADERS, timeout=30)
    response.raise_for_status()
    parser = RobotFileParser()
    parser.parse(response.text.splitlines())
    return {agent: parser.can_fetch(agent, f"https://{site}/") for agent in AGENTS}

rows = {}
for site in NEWS:
    rows[site] = crawler_check(site)
    time.sleep(1)
```

```
allowed = pd.DataFrame(rows).T
print(allowed.map(lambda ok: "allowed" if ok else "blocked").to_string())
```

```
                            * Googlebot   GPTBot ClaudeBot    CCBot Google-Extended
 ↪  PerplexityBot
www.nytimes.com         allowed   allowed  blocked   blocked  blocked         blocked
 ↪  blocked
www.washingtonpost.com  allowed   allowed  allowed   blocked  blocked         allowed
 ↪  blocked
www.wsj.com             blocked   allowed  allowed   blocked  blocked         blocked
 ↪  blocked
www.theguardian.com     allowed   allowed  allowed   blocked  blocked         allowed
 ↪  blocked
www.denverpost.com      allowed   allowed  blocked   blocked  blocked         blocked
 ↪  blocked
coloradosun.com         allowed   allowed  blocked   blocked  blocked         allowed
 ↪  blocked
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Do: a short version of Additional Exercise 4: run `crawler_check()` on a site you use every day that isn't a newsroom. **Share:** the site, and which of the seven agents it blocks from its home page.

```
# Microsprint: run crawler_check() on a site you use every day, and print which agents it blocks
mine = crawler_check("www.reddit.com")
print([agent for agent, ok in mine.items() if not ok])
```

```
['*', 'Googlebot', 'GPTBot', 'ClaudeBot', 'CCBot', 'Google-Extended', 'PerplexityBot']
```

💬 Discuss

- Which newsrooms let GPTBot in but keep ClaudeBot out? What, other than a `Disallow` line, could explain a difference between two AI companies' crawlers?
- Week 1 called the web ”politically contested.” Where in this table do you see the contest, and who is it between?
- We don't appear in `AGENTS` by name. Which column describes what the Wall Street Journal's file asks of us?

Step 6 — Pace yourself, and measure it (10 minutes)

”A good default is one request per second,” the chapter says (Rate Limiting), and if a site's `robots.txt` declares a `Crawl-delay`, respect it. First read the Times's crawl delay with `crawl_delay("*")` and fall back to 1 second, as the chapter does. Then check that a pause does what we think: write `timed_requests(url, n, delay)`, which sends `n` requests with `time.sleep(delay)` after each and records, with `time.time()`, how many seconds after the start each one finished. Send five to `example.com`, the chapter's example site, with no delay and with the polite one, and plot both.

```
# Read rp.crawl_delay("*") into delay; set wait to delay, or to 1.0 if it is None; print both
# Define timed_requests(url, n=5, delay=1.0): note the start time; for each of n requests, send
 ↪  it with HEADERS,
#   record the seconds since the start, then time.sleep(delay); return the list of times
# Run it on https://example.com/ with delay=0 and with delay=wait; plot both lines with labeled
 ↪  axes, a title, and a legend
delay = rp.crawl_delay("*")
wait = delay if delay is not None else 1.0
print("Crawl-delay:", delay, "-> we wait", wait, "seconds")
```

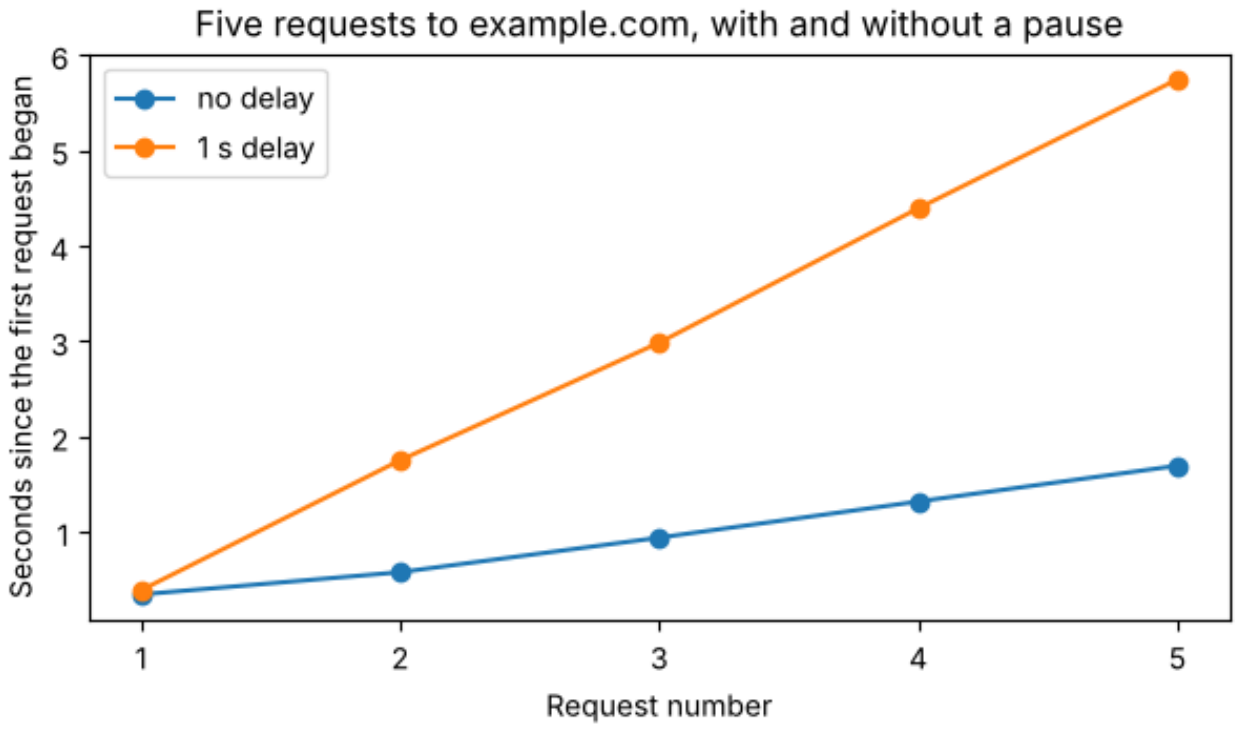
```python
def timed_requests(url, n=5, delay=1.0):
    start = time.time()
    finished = []
    for _ in range(n):
        requests.get(url, headers=HEADERS, timeout=30)
        finished.append(time.time() - start)
        time.sleep(delay)
    return finished

fast = timed_requests("https://example.com/", delay=0)
polite = timed_requests("https://example.com/", delay=wait)
print(f"Five requests: {fast[-1]:.1f} s with no delay, {polite[-1]:.1f} s with {wait:.0f} s
↪  between them")

fig, ax = plt.subplots(figsize=(7, 3.5))
ax.plot(range(1, 6), fast, marker="o", label="no delay")
ax.plot(range(1, 6), polite, marker="o", label=f"{wait:.0f} s delay")
ax.set_xticks(range(1, 6))
ax.set_xlabel("Request number")
ax.set_ylabel("Seconds since the first request began")
ax.set_title("Five requests to example.com, with and without a pause")
ax.legend()
plt.show()
```

```
Crawl-delay: None -> we wait 1.0 seconds
```

```
Five requests: 1.7 s with no delay, 5.7 s with 1 s between them
```



⏱️ Microsprint (4 minutes, alone or in pairs)

Do: a short version of Additional Exercise 3: run `timed_requests()` with a 2-second delay, and measure the gaps between requests. **Share:** how long the five requests took, the average gap between them,

and how far that is from 2 seconds.

```
# Microsprint: run timed_requests() on https://example.com/ with delay=2
# Print the total time, and the average gap between requests (pd.Series(...).diff().mean())
slow = timed_requests("https://example.com/", delay=2)
print(f"{slow[-1]:.1f} s in all, {pd.Series(slow).diff().mean():.2f} s between requests on
 ↪  average")
```

```
9.7 s in all, 2.33 s between requests on average
```

💬 **Discuss**

- Five requests with a one-second pause didn't take five seconds. Where did the difference come from?
- At one request a second, how long would 100,000 pages take? If that's too slow, what would you do instead of going faster?
- In week 1 we saved the pageviews to a CSV so we wouldn't have to ask again. How is saving data a form of rate limiting?

Step 7 — `responsible_get()`: build it once (11 minutes)

Every request from here to the end of the course needs the same things: a User-Agent, a pause, a `timeout`, and a failure that is reported rather than passed on. The chapter wraps them in one function, `responsible_get()` (Building a Responsible Scraper Function); type it as it is there. It turns a 4xx or 5xx into an error with `raise_for_status()`, reports it with `logging`, and returns `None`. Try it on three pages: `example.com`'s home page, a page `example.com` doesn't have, and a Times page that Step 3 found allowed.

```
# Import logging. Define responsible_get(url, headers=None, delay=1.0) as the chapter does:
#   default headers with a User-Agent, updated with headers; get with timeout=30,
 ↪  raise_for_status, log, sleep, return;
#   on an HTTPError log it, sleep, and return None; on any other RequestException log it and
 ↪  return None
# Call it with headers=HEADERS on example.com, example.com/page1, and the Times's
 ↪  /section/technology; print each status or None
import logging

def responsible_get(url, headers=None, delay=1.0):
    """Make an HTTP GET request with responsible defaults.

    Parameters
    ----------
    url : str
        The URL to request
    headers : dict, optional
        Custom headers; a default User-Agent is added if none provided
    delay : float
        Seconds to wait after the request (default: 1.0)

    Returns
    -------
    requests.Response or None
        The response object, or None if the request failed
    """
    default_headers = {
        "User-Agent": "WebDataScience/1.0 (student-project; your-email@colorado.edu)"
    }
    if headers:
```

```
        default_headers.update(headers)

    try:
        response = requests.get(url, headers=default_headers, timeout=30)
        response.raise_for_status()  # Treat 4xx/5xx statuses as failures
        logging.info(f"{response.status_code} {url}")
        time.sleep(delay)
        return response
    except requests.exceptions.HTTPError as e:
        logging.error(f"HTTP error: {e}")
        time.sleep(delay)  # A failed request still counts against the server
        return None
    except requests.exceptions.RequestException as e:
        logging.error(f"Request failed ({type(e).__name__}): {url}")
        return None

for url in ["https://example.com/", "https://example.com/page1",
 ↪  "https://www.nytimes.com/section/technology"]:
    response = responsible_get(url, headers=HEADERS)
    print(url, "->", response.status_code if response is not None else None)
```

```
ERROR:root:HTTP error: 404 Client Error: Not Found for url: https://example.com/page1

https://example.com/ -> 200

https://example.com/page1 -> None

ERROR:root:HTTP error: 403 Client Error: Forbidden for url:
 ↪  https://www.nytimes.com/section/technology

https://www.nytimes.com/section/technology -> None
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Do: write `polite_get(url, parser)`, which checks `robots.txt` before it sends anything, and try it on the Times's search page and home page with Step 3's `rp`. **Share:** which page was skipped, which was sent, and what came back.

```
# Microsprint: define polite_get(url, parser): if parser.can_fetch("*", url) is False, print that
 ↪  you skipped it
#   and return None without sending anything; otherwise return responsible_get(url,
 ↪  headers=HEADERS)
# Try it on https://www.nytimes.com/search?query=robots and https://www.nytimes.com/, with Step
 ↪  3's rp
def polite_get(url, parser):
    if not parser.can_fetch("*", url):
        print("skipped, robots.txt disallows it:", url)
        return None
    return responsible_get(url, headers=HEADERS)

for url in ["https://www.nytimes.com/search?query=robots", "https://www.nytimes.com/"]:
    response = polite_get(url, rp)
    print(url, "->", response.status_code if response is not None else None)
```

```
skipped, robots.txt disallows it: https://www.nytimes.com/search?query=robots
https://www.nytimes.com/search?query=robots -> None

ERROR:root:HTTP error: 403 Client Error: Forbidden for url: https://www.nytimes.com/
```

```
https://www.nytimes.com/ -> None
```

💬 Discuss

- The 404 and the 403 printed a log line, but the 200 printed nothing. Why? (Look at which `logging` function each one calls.)
- The Times's `robots.txt` allows its section pages for `"*"`, yet its server answered 403. Which signal do you follow, and what does the chapter say to do?
- Week 1 blamed a 403 on a missing User-Agent. We sent one. What else can a 403 mean?

Step 8 — Five questions before you collect (10 minutes)

No code can tell you whether a collection is right. The chapter's ethical decision framework asks five questions before any project: legality, consent, proportionality, privacy, and server impact. Answer them for what we collected today, and keep the answers with the code, the way you'd keep a lab notebook: one sentence each, in a dictionary, printed as a record. Then try the framework on a harder case.

```
# Make decisions, a dict: its keys are the chapter's five questions (Legality, Consent,
 ↪  Proportionality, Privacy, Server impact),
#   its values one sentence each about what we collected today
# Print each question and its answer, one per line
decisions = {
    "Legality": "robots.txt files and a few public pages, fetched without logging in; no Terms of
     ↪  Service accepted",
    "Consent": "robots.txt is written to be read by machines, so reading it is the use it
     ↪  invites",
    "Proportionality": "we kept only allowed/blocked answers and timings, not page content",
    "Privacy": "no personal data: the files name crawlers and paths, not people",
    "Server impact": "about thirty requests to a dozen sites, never more than five in a row
     ↪  without a pause",
}
for question, answer in decisions.items():
    print(f"{question}: {answer}")
```

```
Legality: robots.txt files and a few public pages, fetched without logging in; no Terms of
 ↪  Service accepted
Consent: robots.txt is written to be read by machines, so reading it is the use it invites
Proportionality: we kept only allowed/blocked answers and timings, not page content
Privacy: no personal data: the files name crawlers and paths, not people
Server impact: about thirty requests to a dozen sites, never more than five in a row without a
 ↪  pause
```

⏱️ Microsprint (6 minutes, in pairs)

Do: a short version of Additional Exercise 2: a researcher wants to scrape user profiles from a dating app to study self-presentation across demographic groups. Answer the five questions in a comment line each, then recommend. **Share:** your recommendation in one sentence, and the question that decided it.

```
# Microsprint: the dating-app study, one comment line per question, then your recommendation
# Legality: profiles sit behind a login, so the researcher has accepted the app's Terms; after
 ↪  hiQ, contract risk is high
# Consent: users wrote profiles for possible matches, not for researchers; they had no say in
 ↪  this use
# Proportionality: self-presentation could be studied with fewer fields, and without photos or
 ↪  locations
# Privacy: photos, ages, places, and orientations make users easy to re-identify; a breach could
 ↪  out people
```

```
# Server impact: small next to the other four, but logged-in scraping is easy for the app to
 ↪  trace to an account
# Recommendation: don't scrape; recruit consenting users (data donation) under IRB review.
 ↪  Consent and privacy decide it.
```

💬 **Discuss**

- The chapter says scraping becomes human subjects research when it collects data about identifiable people to produce generalizable knowledge. Does the dating-app study need an IRB, and did ours?
- If some of the app's users live in the EU, what does GDPR add to your answer, wherever the researcher is?
- Week 1's scientific norms asked us to document our methods. Where would `decisions` belong in a paper or a repository?

Step 9 — What did we learn? (5 minutes)

In a few sentences, as comments:

- In Step 5, how many of the six newsrooms let GPTBot fetch their home page, and how many let ClaudeBot? What would explain the difference?
- Twice today a tool gave a wrong answer without warning: in Step 3 (in older Pythons) and in Step 4. How did we catch each one?
- In Step 6 and its microsprint, how long did five requests take with each delay? Did the pause do what we intended?
- In Step 7, which page did `robots.txt` allow but the server refuse? What should a responsible scraper do next?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday):

- Step 1, a `HEADERS` dictionary that names the course and your email: Step 0's `HEADERS`, with the course added.
- Step 2, fetch Wikipedia's `robots.txt` and print its status and start: Step 2 fetched the Times's file the same way.
- Step 3, parse it with `rp.parse()` and test three paths: Step 3 parsed the Times's file; Step 4 showed why not `rp.read()`.
- Step 4, two more sites of your choice, fetched politely and counted: Step 5's loop, with `time.sleep(1)`; Step 2 took a file apart line by line.
- Step 5, one DataFrame for all three sites, with `crawl_delay("*")`: Step 5 built a DataFrame from a dict per site; Step 6 read a crawl delay.
- Step 6, interpret: Steps 2 and 5, and their Discuss questions about what a file says of the organization behind it.

```
# Your answers here, as comments
# Step 5: three of the six newsrooms let GPTBot fetch their home page (the Post, the Journal, the
 ↪  Guardian),
#   and none let ClaudeBot. Those three have announced licensing deals with OpenAI; the Times,
 ↪  which blocks GPTBot,
#   is suing OpenAI (the chapter's case). The file records business deals, not only technical
 ↪  preferences.
# Step 3: the parser said /svc/wordle is allowed, because "Allow: /svc/wordle" (line 75) is
 ↪  longer than
#   "Disallow: /svc" (line 70); Python before 3.14.5 obeyed line 70 and said blocked. We caught
 ↪  it by reading the file.
#   Step 4: read() got a 403 for "Python-urllib/3.14", set disallow_all to True, and said an
 ↪  ordinary article
#   was off limits. We caught it by sending the same request ourselves and seeing the 403.
# Step 6: five requests took 1.7 s with no pause, 5.7 s with 1 s, and 9.7 s with 2 s (2.33 s
 ↪  apart on
```

```
#   average). The pause sets a floor, and each request's own time comes on top of it.
# Step 7: the Times allows "*" on /section/technology and its home page, but answered both with a
 ↪  403 to our
#   named User-Agent. Respect it: don't retry or disguise the client; ask for permission or find
 ↪  an API.
```

Friday: from the lab to the textbook (15 minutes)

What did the lab turn up that the chapter should know? Some places to look:

- **Step 3:** the chapter never says that `can_fetch()`'s answers depend on the Python version. Since 3.14.5 the parser follows RFC 9309's longest-match rule; before, it obeyed the first matching rule. Should the chapter say so, and where?
- **Step 2:** the chapter's comparison counts 359 lines in the Times's `robots.txt`. How many did you get? A count that drifts is a reason to date it, or to show how to find it rather than state it.
- **Step 7:** `responsible_get()` logs its successes with `logging.info()`, which a notebook doesn't show by default. Did that confuse you? What one line would make the chapter's function show them?
- **Step 5:** would the AI-crawler table make a good Additional Exercise? Write it as one.
- Any cell that failed for you, or a sentence in the chapter you had to read twice.

The revision framework says how to propose a change and what it needs to be accepted.